KỸ THUẬT LẬP TRÌNH / JAVA

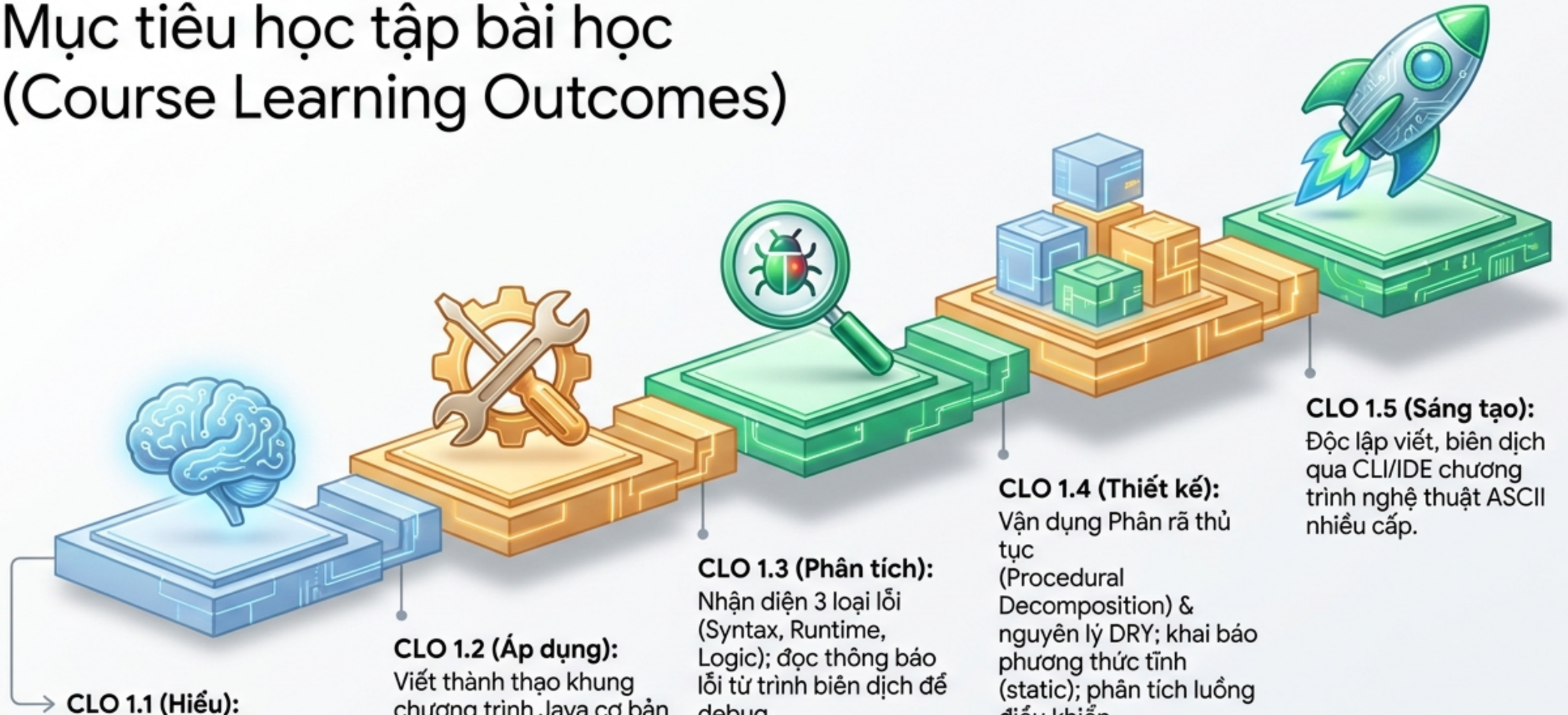
Chương 1: Introduction to Java Programming

- Dành cho sinh viên Khóa 2026 – Lớp IT26M
- Khoa Công nghệ Thông tin
- Dựa trên giáo trình: Building Java Programs
- Buổi học tích hợp Lý thuyết & Thực hành tại lớp (Thời lượng: 3 Tiết)
- Năm học 2026



Gemini Notebook

Mục tiêu học tập bài học (Course Learning Outcomes)



CLO 1.1 (Hiểu):
Giải thích bản chất kiến trúc máy tính, hệ nhị phân, cơ chế biên dịch vs thông dịch, vai trò máy ảo JVM và mã Bytecode (WORA).

CLO 1.2 (Áp dụng):
Viết thành thạo khung chương trình Java cơ bản (class, main), phân biệt `print/println`, sử dụng ký tự thoát (`\n`, `\t`, `\t`, `"`, `\\`).

CLO 1.3 (Phân tích):
Nhận diện 3 loại lỗi (Syntax, Runtime, Logic); đọc thông báo lỗi từ trình biên dịch để debug.

CLO 1.4 (Thiết kế):
Vận dụng Phân rã thủ tục (Procedural Decomposition) & nguyên lý DRY; khai báo phương thức tĩnh (static); phân tích luồng điều khiển.

CLO 1.5 (Sáng tạo):
Độc lập viết, biên dịch qua CLI/IDE chương trình nghệ thuật ASCII nhiều cấp.

Gemini Notebook

Lộ trình buổi học 3 tiết (Tích hợp Lý thuyết & Thực hành)

TIẾT 1 (50 phút) - Nền tảng & Khởi đầu

- Khái niệm tính toán & Nhị phân
- Compiler vs. Interpreter
- Kiến trúc JVM & Bytecode
- Quy trình: Edit-Compile-Run
- Chương trình Hello.java

[Lab 1]: Viết & chạy Java trên CLI và IDE.

TIẾT 2 (50 phút) - Cú pháp & Gỡ lỗi

- Ký tự thoát (Escape Seq)
- Định danh, Từ khóa, CamelCase & Chú thích
- Phân loại 3 nhóm lỗi (Syntax, Runtime, Logic)
- Chiến lược đọc lỗi & gỡ lỗi

[Lab 2]: Thử thách Debug & Escape Strings.

TIẾT 3 (50 phút) - Phân rã & Case Study

- Tư duy phân rã Top-down & Phương thức tĩnh (static)
- Luồng điều khiển (Control Flow) & Nguyên lý DRY
- Case Study DrawFigures (Naïve -> Cấp 1 -> Triệt để)

[Lab 3]: Mini Project vẽ Tàu

Gemini Notebook

TIẾT 1: NỀN TẢNG HỆ THỐNG TÍNH TOÁN & KHỞI ĐẦU

Tại sao chúng ta cần học lập trình? (Why Programming?)

> Người ta thường nói rằng một người không thực sự hiểu điều gì đó cho đến khi họ dạy nó cho người khác. Trên thực tế, một người không thực sự hiểu điều gì đó cho đến khi họ dạy nó cho một chiếc máy tính, tức là diễn đạt nó dưới dạng một thuật toán.
>
> **— Donald Knuth**



Máy tính

Thiết bị phần cứng vạn năng (General-purpose device).

Chương trình (Program)

Tập hợp các chỉ thị ra lệnh cho phần cứng.

Lập trình

Rèn luyện tư duy giải quyết vấn đề có cấu trúc.

Gemini Notebook

Thành phần cốt lõi của máy tính

CPU: Bộ vi xử lý trung tâm. "Bộ não" thực thi các phép tính logic và số học.



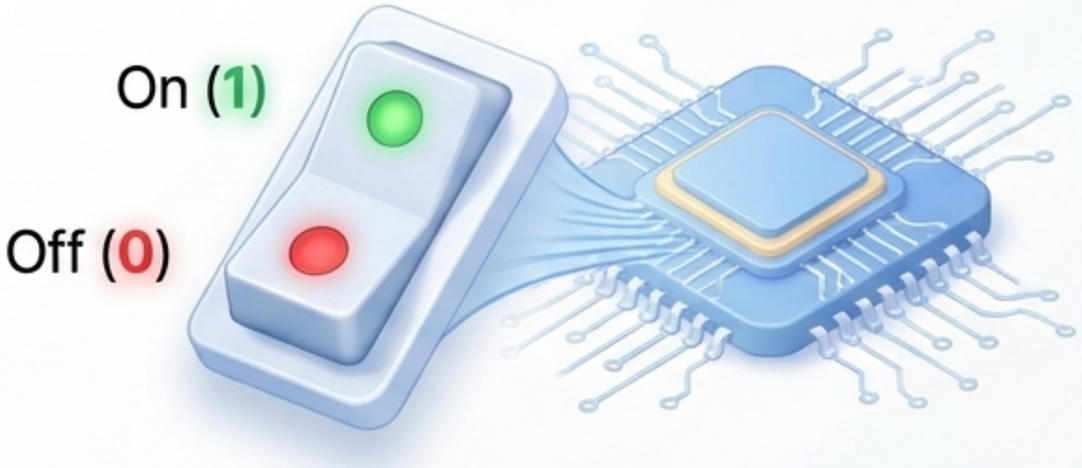
RAM: Bộ nhớ chính. Lưu trữ tạm thời, tốc độ cực nhanh, mất dữ liệu khi mất nguồn.

SSD/Ổ cứng: Bộ nhớ thứ cấp. Lưu trữ dữ liệu vĩnh viễn (persistent storage).

Thiết bị I/O: Bàn phím, chuột, màn hình, kết nối mạng. Giao tiếp với thế giới bên ngoài.

Gemini Notebook

Thế giới số & Biểu diễn dữ liệu bằng Hệ nhị phân (Binary)



Cơ chế biểu diễn số theo cơ số 2:

Trọng số	$2^3=8$	$2^2=4$	$2^1=2$	$2^0=1$
Bit	**1**	**0**	**1**	**1**

$(\mathbf{1011})_2 = 1\times8 + 0\times4 + 1\times2 + 1\times1$
$= 8 + 0 + 2 + 1 = \mathbf{11}$ (Hệ thập phân)

- Tại sao máy tính sử dụng số nhị phân? Phần cứng cấu tạo từ hàng tỷ bóng bán dẫn (transistors).
- Bật (On) / Điện áp cao = **1**.
- Tắt (Off) / Điện áp thấp = **0**.
- **Bit**: Đơn vị nhỏ nhất (0/1). **Byte**: 8 bits = 256 trạng thái (0-255).

Chốt lại: Mọi chương trình, hình ảnh, video cuối cùng đều được ánh xạ về dãy bit 0 và 1!

Gemini Notebook

Cơ chế thực thi: Trình biên dịch vs. Thông dịch

Trình biên dịch truyền thống (C/C++) (Compiler)



– **Dịch** toàn bộ mã nguồn thành Mã máy trực tiếp (Native Machine Code).
– **Ưu điểm:** Tốc độ thực thi cực nhanh, tối ưu phần cứng.
– **Nhược điểm chí tử:** Phụ thuộc nền tảng! (File Windows không chạy trên macOS/Linux).

Trình thông dịch (Python, JavaScript) (Interpreter)

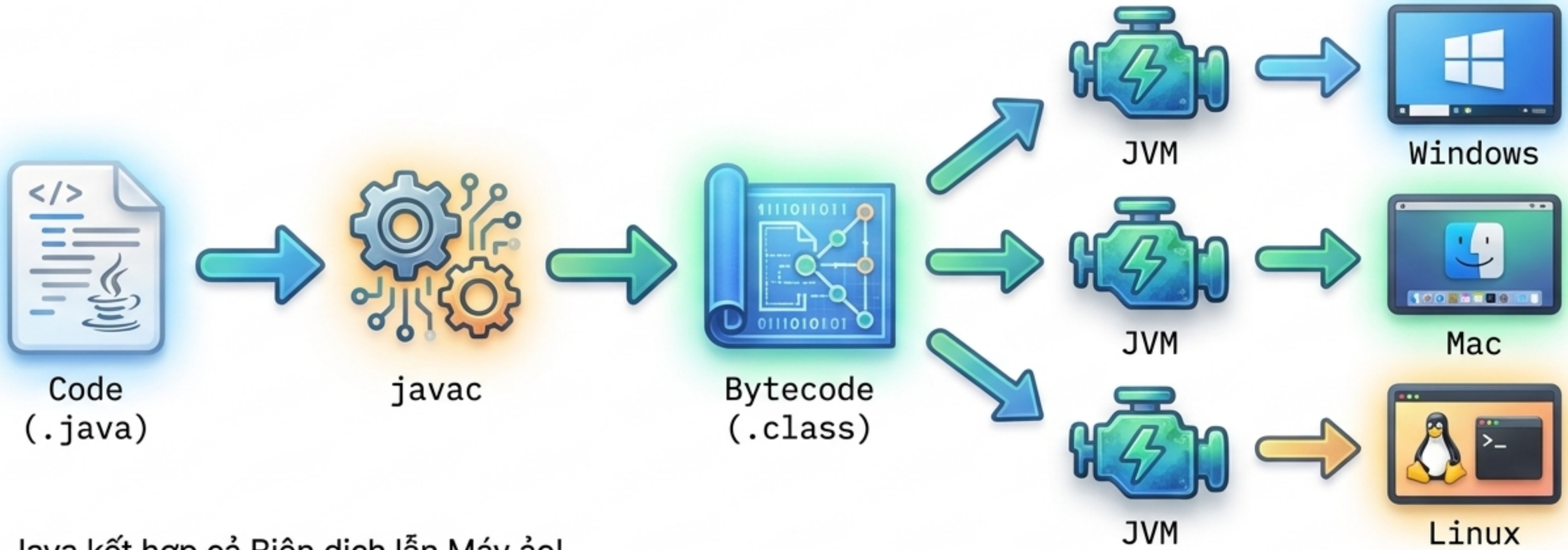


– Đọc và diễn giải trực tiếp từng dòng lệnh tại thời điểm chạy.
– **Ưu điểm:** Linh hoạt, dễ thử nghiệm trực tiếp.
– **Nhược điểm:** Tốc độ thực thi chậm hơn nhiều so với mã đã biên dịch sẵn.

Câu hỏi đặt ra cho Java năm 1995: Làm sao vừa đạt tốc độ thực thi cao, vừa chạy trên mọi nền tảng phần cứng mà không cần biên dịch lại?

Gemini Notebook

Triết lý: “Write Once, Run Anywhere” (WORA)

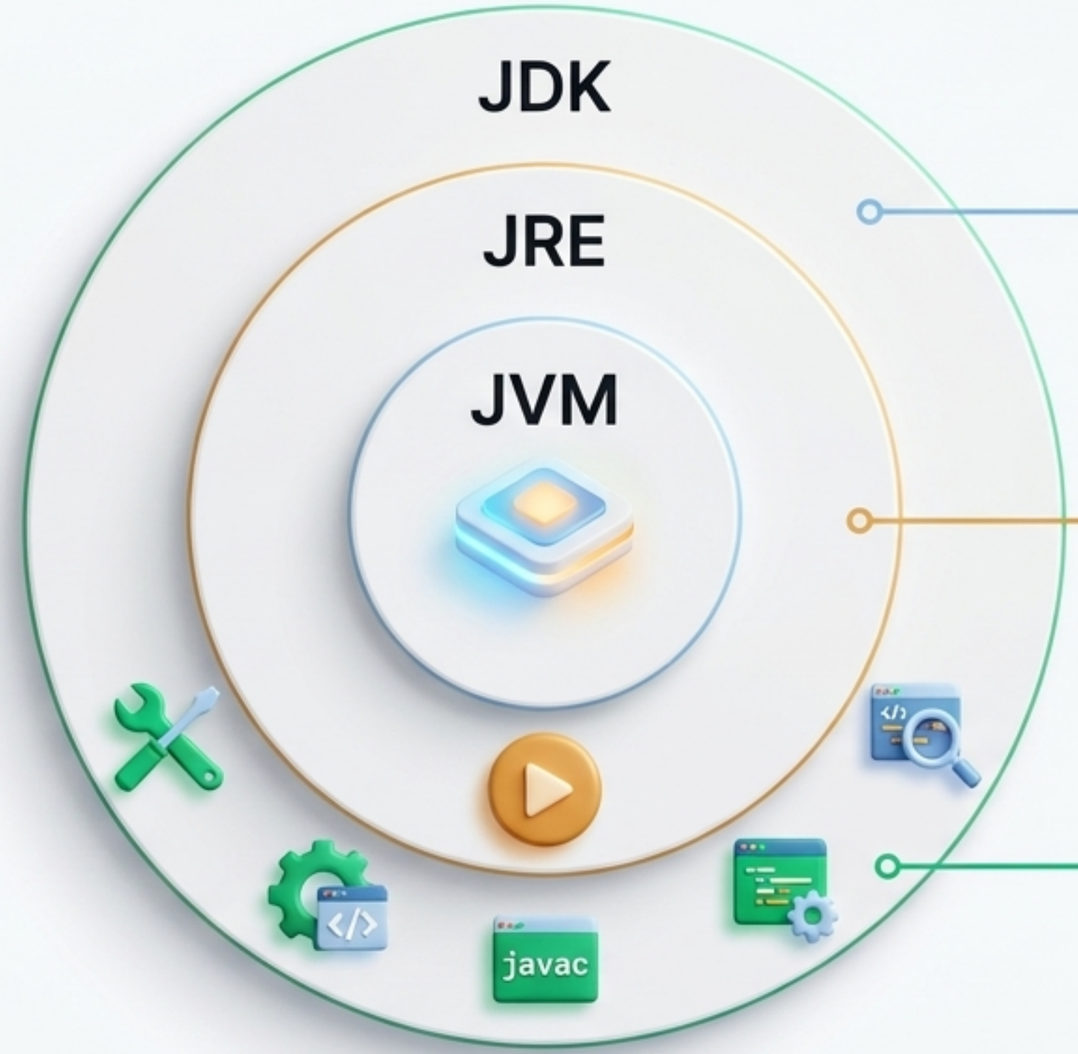


- Java kết hợp cả Biên dịch lẫn Máy ảo!
- Trình biên dịch (javac) không dịch ra mã máy thật. Nó dịch mã nguồn (.java) thành Java Bytecode (.class).
- Bytecode là tập lệnh chuẩn cho một cỗ máy lý thuyết gọi là Java Virtual Machine (JVM).
- Mỗi hệ điều hành có một bản JVM riêng để thông dịch/JIT-compile Bytecode ra mã máy phần cứng.

Gemini Notebook

Bộ công cụ Java:
Phân biệt JDK, JRE, JVM

`JDK = JRE + Tools`
`⊃ JRE = JVM + Libs`



Java Virtual Machine:
Môi trường thực thi Bytecode.

Java Runtime Environment:
Gồm JVM + Thư viện chuẩn (Java Class Libraries) để người dùng chạy ứng dụng.

Java Development Kit:
Dành cho lập trình viên!
Chứa JRE + Trình biên dịch (`javac`) + Công cụ gỡ lỗi.

Gemini Notebook

Quy trình phát triển chương trình Java: Edit → Compile → Run



Bước 1: Soạn thảo (Edit)

Viết mã nguồn vào file văn bản thuần.
Bắt buộc đuôi file: `Hello.java`.



Bước 2: Biên dịch (Compile)

Lệnh: `javac Hello.java`

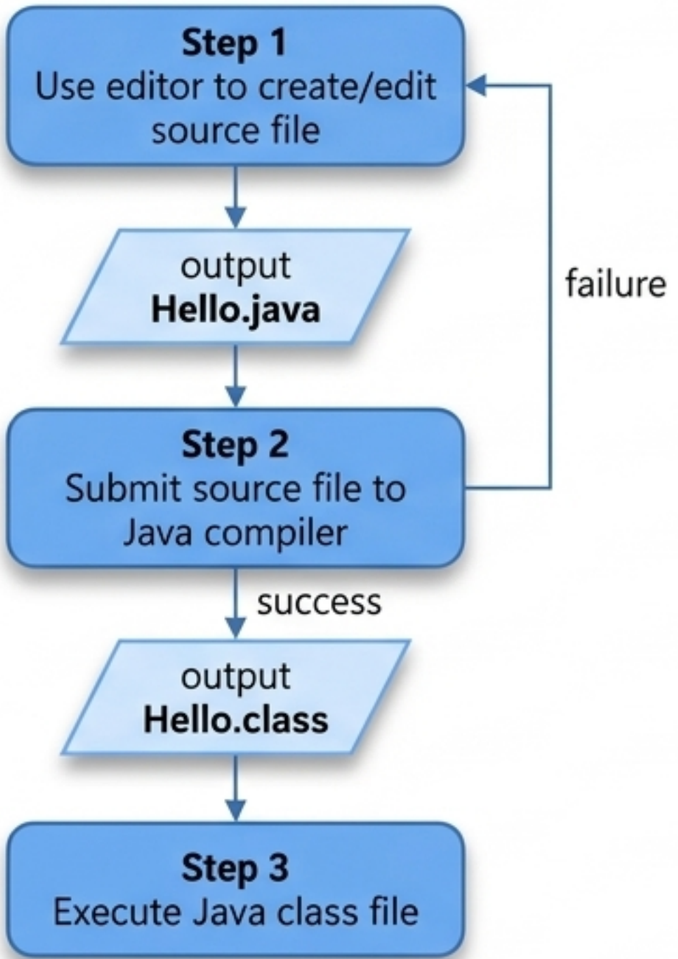
Kiểm tra lỗi cú pháp. Tạo ra file bytecode: `Hello.class`.



Bước 3: Thực thi (Run)

Lệnh: `java Hello`

Khởi động máy ảo JVM. JVM nạp bytecode, chuyển cho CPU thực thi. Kết quả xuất hiện trên màn hình!



Hình 1.1: *Quá trình tạo và thực thi chương trình Java*

Gemini Notebook

Chương trình Java hoàn chỉnh đầu tiên

Cấu trúc mẫu:
public class <TênLớp> { ... }

```java
// Chuong trinh Java dau tien cua lop IT26M
public class Hello
{
    public static void main(String[] args)
    {
        System.out.println("Hello, world!");
    }
}
```

Quy tắc vàng 1 (Ngoặc nhọn { }): Mọi mã nguồn phải nằm bên trong một lớp (class). Đóng gói phạm vi (scope). Mọi { mở phải có } đóng tương ứng.

Quy tắc vàng 2 (Tên file): Tên file bắt buộc trùng khớp 100% với tên lớp công khai. Lớp `public class Hello` ⇔ File `Hello.java`.

Quy tắc vàng 3: Java phân biệt chữ hoa và chữ thường (*case-sensitive*).

Gemini Notebook

Mổ xẻ phương thức **main** – Điểm bắt đầu của mọi ứng dụng

```
public static void main(String[] args) { … }
```



public | **Quyền truy cập công khai**. Bất kỳ đối tượng nào (bao gồm JVM) cũng có thể gọi phương thức này.



static | **Phương thức tĩnh cấp lớp.** Thực thi trực tiếp từ lớp mà không cần khởi tạo đối tượng (instance).



void | **Kiểu dữ liệu trả về.** Phương thức thực hiện công việc và không trả lại giá trị nào.



main | **Tên phương thức.** Tên quy ước đặc biệt mà JVM tìm kiếm đầu tiên khi chạy chương trình.



String[] args | **Mảng tham số dòng lệnh.** Cho phép truyền các chuỗi đối số từ bên ngoài khi khởi động chương trình.

Gemini Notebook

Lệnh xuất dữ liệu: println vs. print

`System.out.println(...)`: In nội dung ra console và tự động xuống dòng mới (newline).

```
System.out.println("Dong thu nhat");
System.out.println("Dong thu hai");
```

```
Dong thu nhat
Dong thu hai
```



`System.out.print(...)`: In nội dung ra console và giữ nguyên con trỏ ở cuối dòng hiện tại.

```
System.out.print("Xin ");
System.out.print("chao ");
System.out.println("IT26M!");
```

```
Xin chao IT26M!
```



Lưu ý quan trọng! Mọi câu lệnh trong Java bắt buộc kết thúc bằng dấu chấm phẩy `;`. Lệnh `System.out.println()` không có tham số tạo ra một dòng trống.

Gemini Notebook

[CHECKPOINT 1] & [HANDS-ON LAB 1]

Câu 1: Lệnh hợp lệ?

✅ **Đáp án C:** System.out.println("Hello"); (Java phân biệt hoa thường, chuỗi trong nháy kép "")

Câu 2: Vai trò file .class?

✅ **Đáp án B:** Chứa mã Bytecode đã được biên dịch để JVM thực thi độc lập nền tảng.

HANDS-ON LAB 1: 15 Phút

1. Mở Terminal, gõ **javac -version** & **java -version**.
2. Tạo thư mục Lab1 & file **WelcomeIT26M.java**.
3. Biên dịch: **javac WelcomeIT26M.java**
4. Chạy: **java WelcomeIT26M**

```java
public class WelcomeIT26M {
  public static void main(String[] args) {
    System.out.println("Chao mung lop IT26M den voi Java!");
    System.out.println("Viet mot lan, chay moi noi (WORA).");
  }
}
```

Gemini Notebook